

Computer Graphics

🕒 Created time	August 1, 2026 2:21 PM
🕒 Last edited time	August 4, 2026 8:20 PM
🏷️ Tag	Year 4 Term 2

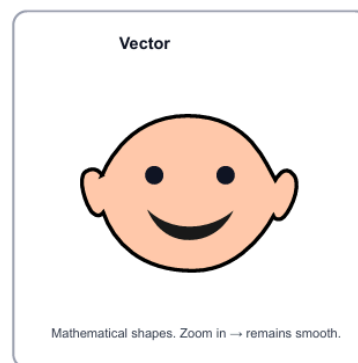
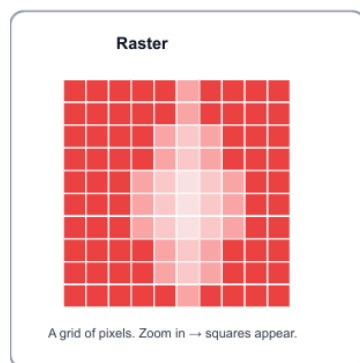
Introduction to Computer Graphics

Definition

- Computer graphics is the field of computer science concerned with **creating, storing, manipulating and displaying visual content** using computers.
- Computer graphics is the use of **computation** to **generate** or control **image**

Vector vs Raster Graphics

Feature	Raster Graphics	Vector Graphics
Definition	A raster image is a rectangular grid of small picture elements called pixels.	A vector image describes shapes mathematically using geometric primitives
Representation	Pixel grid	Mathematical shapes
Scalability	Loses quality when enlarged	Stays sharp at any size
Zoom effect	Blocky/pixelated	Smooth/clean
Best for	Photos, textures, detailed images, medical images, satellite image	Logos, icons, diagrams
File types	JPG, PNG, GIF	SVG, EPS, PDF



Pixel

- A pixel (short for picture element) is the **smallest unit** of a digital image or display

Coordinate

- A coordinate is **numerical** value that **specifies the position of a point** in a defined space

- It **represent a single point** in a **raster** image and carries **color/intensity** information
- Formally, a pixel can be thought of as a **discrete sample in a 2D grid** that approximates a continuous image.
- In computer graphics, coordinates are usually expressed in a **Cartesian** system as (x, y) for $2D$ image,
- They provide a way to **locate pixels (in raster graphics) or control points** within the image plane,

Color as numbers

Color on a digital display is usually represented numerically. A common model is the *RGB* color model, where colors are **produced by mixing Red, Green and Blue light**.

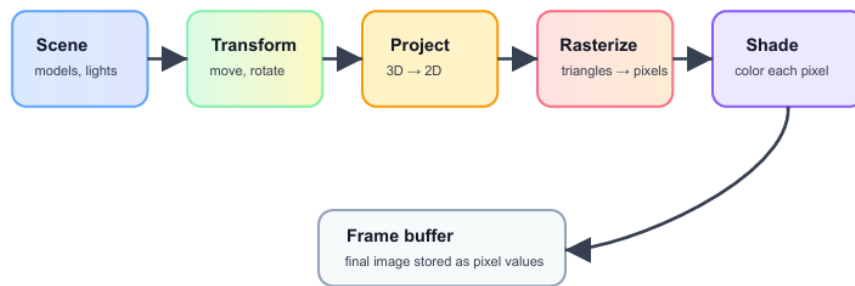
- Each color rang from 0 to 255
- Once pixel consists of these 3 color
 - So, one pixel can be represent by $256^3 = 16,777,216$ possible colors
 - This is often called **24 bit color or true color**

Aspect	Screen Color (RGB)	Paint Color (CMYK)
Basis	Light emission	Pigment absorption
Process	Additive, red, green, and blue light are added together.	Subtractive, pigments absorb some wavelengths and reflect others.
Primary components	Red, Green, Blue	Cyan, Magenta, Yellow (+ Black)
Mixing result	All = White	All = Black/Dark
Usage	Digital displays	Printing, painting

Aspect	Modeling	Rendering
Definition	The process of creating a mathematical or geometric representation of objects	The process of generating a final image from a model.
Focus	Defines the shape, structure, and position of objects in a scene.	Converts the mathematical description into a visual representation .
Include	object shape, object size, position, orientation, material, surface color, texture etc.	camera position, projection type, light sources, material response, visibility, shadows etc.
Output	Mathematical description	Final image
Tools	CAD, 3D modeling software	Rendering engines
Stage	Scene creation	Scene visualization

The graphics pipeline

A graphics pipeline is a sequence of steps that converts scene description into final pixel.



1. Scene data : The scene contains **objects, lights, cameras and material** information.
2. Transformation: Objects are *moved, rotated, scaled, or positioned in the scene*. Transformations are usually represented using **matrices**.
3. Projection: A 3D scene is converted **into a 2D view**.
4. Rasterization: **Geometric shapes** such as lines and triangles are converted into **pixels**.
5. **Shading**: The color of each is computed. Shading may consider material color, light direction, texture maps, surface normals and shadows.
6. **Frame buffer**: The generated pixel values are stored in a memory area called the frame buffer.

Attribute	Vector CRT	Raster CRT	Flat-Panel (LCD/OLED)
Definition	A display where the electron beam directly traces geometric shapes (lines, curves) instead of scanning pixel grids.	A display where the electron beam sweeps row by row to illuminate pixels, forming a complete raster image.	A display that uses a matrix of controllable pixels (liquid crystals or organic LEDs) to produce images without bulky electron beams .
Technology Basis	Beam draws lines	Beam scans rows	Fixed pixel grid
Best For	Wireframe drawings	Images, text, video	Modern screens, high-res visuals
Limitation	Not good for filled images	Bulky, outdated	Cost, OLED burn-in

Image Representation

An image is a two-dimensional function $f(x, y)$ where

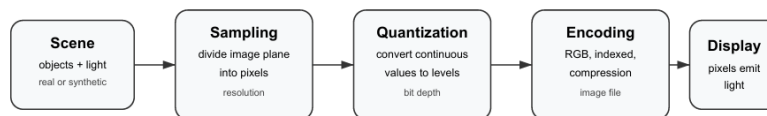
- x and y are spatial coordinates
- The value of $f(x, y)$ at any point represents the intensity (brightness or color) at that coordinate

An image is a matrix of meaningful numbers.

Raster image

- A raster image is a rectangular arrangement of small elements called pixels.
- A pixel is the smallest addressable unit of a raster image.
 - It has a **position** and **one or more stored values**

$$\text{Number of pixels} = W \times H$$



1. Sampling:

- a. Sampling means measuring a **continuous signal at discrete positions**
 - i. Sampling decide where we measure the scene
- b. If sampling is too low
 - i. fine details may disappear
 - ii. produce aliasing artifacts

2. Quantization

- a. Quantization means **converting a continuous value into one of a finite numbers of levels**
 - i. Example: 256 brightness level
- b. If quantization is too coarse,
 - i. smooth gradients may become visible steps
 1. Known as **Banding**

Storing Pixel Color : Direct Coding vs Lookup tables

Attribute	Direct Color	Lookup Table (palette-based color)
Definition	Each pixel directly stores its full color value.	Each pixel stores an index that points to a color in a predefined table (palette).
Storage	Requires more memory since every pixel holds full color info.	Requires less memory since pixels only store small indices.
Flexibility	Can represent millions of colors (true color).	Limited to the size of the palette
Speed	Faster to render because no lookup is needed.	Slightly slower due to table lookup for each pixel.

Attribute	Direct Color	Lookup Table (palette-based color)
Usage	Modern displays, high-quality images, photographs.	Early graphics systems, GIF images, icons, low-memory devices.

Bit Depth

Bit depth describes how many bits are used to represent each pixel or each channel.

x bits = 2^x levels

Higher bit depth matters: medical imaging, HDR photography, cinema production, scientific visualization, image editing, physically based rendering.

Display monitors

A monitor converts digital image into visible light.

Name	Resolution
HD	1280 × 720
Full HD	1920 × 1080
QHD	2560 × 1440
4K UHD	3840 × 2160
8K UHD	7680 × 4320

Pixel density

Pixel density measures how many pixels are **packed into a physical length**, often expressed as *PPI: pixels per inch*.

Image File formats

Format	Definition	Key Attributes	Advantages	Limitations	Typical Uses
BMP (Bitmap)	A raster image format that stores pixel data directly , often uncompressed .	Large file sizes, simple structure.	Easy to read/write, widely supported, easy to understand, useful for teaching, low-level processing	Not efficient for storage or transfer, not for web distribution.	Windows system images, raw pixel storage.
JPEG (Joint Photographic Experts Group)	A lossy compressed format for	24-bit color, adjustable compression .	High compression ratio, good for	Lossy — repeated saves degrade	Digital photography, web images.

Format	Definition	Key Attributes	Advantages	Limitations	Typical Uses
	continuous-tone images.		photos, very small file, widely supported	quality, compression artifacts, poor for sharp text, diagrams	
PNG (Portable Network Graphics)	Lossless compressed raster format with transparency support .	24-bit color + alpha channel.	High quality , supports alpha transparency , lossless . good for diagrams , icons, preserve sharp edges	Larger than JPEG for photos.	Web graphics, UI design, images needing transparency.
GIF (Graphics Interchange Format)	Indexed color format supporting up to 256 colors and simple animations .	Uses palette-based compression (LZW).	Small size , supports animation and transparency , wide compatibility	Limited color range, not suitable for photos, not high quality	Web graphics, icons, simple animations.
SVG (Scalable Vector Graphics)	XML-based vector format for 2D graphics, it stores shapes, paths, text and styling rather than a fixed pixel grid.	Resolution-independent , text-based.	Infinitely scalable , editable , small size, excellent for diagrams, icons	Not suitable for complex photos.	Logos, icons, diagrams, web graphics.
Extra:					
TIFF (Tagged Image File Format)	Flexible format supporting multiple compression schemes and high bit depth.	Can be lossless or lossy, supports layers.	Very high quality, widely used in publishing.	Large file sizes, less common on web.	Printing, scanning, professional photography.
RAW	Unprocessed image data from digital cameras.	Contains sensor data, very large files.	Maximum detail, flexible for editing.	Requires special software, huge size.	Professional photography, image editing.
WEBP	Modern format developed by Google for web images.	Supports both lossy and lossless compression, transparency, and animation.	Smaller file sizes than JPEG/PNG, versatile.	Limited support in older systems.	Web graphics, optimized online images.

Gamma Correction and Perception

Gamma correction is used to manage the **non-linear relationship between stored pixel values and displayed brightness**.

Mandelbrot Set

- **computer visualization** to reveal its infinite, intricate patterns.
- Used to generate stunning fractal images.

The Mandelbrot set is the set of all complex numbers c for which the sequence defined by,

$$z_{n+1} = z_n^2 + c, z_0 = 0$$

does not diverge to infinity.

Scan Conversion

Every scan conversion algorithm takes a **continuous geometric description** and turns it into a **finite set of pixels** that looks visually correct.

Circle scan conversion

A mathematical circle centered at the origin with radius r is described by

$$x^2 + y^2 = r^2$$

This equation describes **infinitely many points**. A raster display, however, needs a **finite set of pixels**.

A naive approach would be to solve for y as,

$$y = \pm\sqrt{r^2 - x^2}$$

where,

1. square root is **computationally expensive**
2. **floating-point arithmetic is slower** than simple integer updates
3. we want a method that picks the next pixel using only **small local decisions**

Symmetric of a circle

$$(x, y), (y, x), (-x, -y), (-x, y), (x, -y), (-y, x), (y, -x), (-y, -x)$$

Midpoint circle algorithm [Midpoint Circle Drawing Algorithm Explained | PDF | Algorithms And Data Structures | Theoretical Computer Science](#)

$$E = (x + 1, y)$$

$$SE = (x + 1, y - 1)$$

$$M = mid = (x + 1, y - \frac{1}{2})$$

M is inside the circle, $f(m) < 0 : E$

Else $f(M) > 0 : SE$

Initial : $(x_0, y_0) = (0, r)$

Update Rules

$x = 0, y = r, p = 1 - r$

$x \leq y :$

$plot8(x, y)$

$x = x + 1$

$p < 0 \rightarrow p = p + 2x + 1$

else $\rightarrow y = y - 1, p = p + 2x + 1 - 2y$

Bresenham's circle algorithm

$x = 0, y = r, d = 3 - 2r$

while $x \leq y :$

$plot8(x, y)$

if $d < 0 :$

$d = d + 4x + 6$

else :

$d = d + 4(x - y) + 10$

$y = y - 1$

$x = x + 1$

Ellipse Scan : midpoint ellipse algorithm

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

a = semi-major axis

b = semi-minor axis

The midpoint ellipse algorithm divides the first quadrant into two regions.

$\left| \frac{dx}{dy} \right| < 1 \rightarrow$ The curve changes more slowly in y than in x , so we mainly advance in the x -direction.

$\left| \frac{dx}{dy} \right| > 1 \rightarrow$ The curve changes more rapidly in y , so we mainly advance in y -direction.

Filled Rectangle

$x_{\min}, y_{\min}, x_{\max}, y_{\max}$

for $y = y_{\min}$ to $y_{\max} :$

for $x = x_{\min}$ to $x_{\max} :$

$plot(x, y)$

Region Filling

BoundaryFill4(x,y,fillcolor, boundaryColor) :

if getPixel(x,y) is not boundaryColor and not fillColor:

setPixel(x,y,fillColor)

BoundaryFill4(x+1,y,fillcolor, boundaryColor)

BoundaryFill4(x-1,y,fillcolor, boundaryColor)

BoundaryFill4(x,y+1,fillcolor, boundaryColor)

BoundaryFill4(x,y-1,fillcolor, boundaryColor)

Flood fill

FloodFill4(x, y, oldColor, newColor):

if getPixel(x, y) == OldColor:

setPixel(x, y, newColor)

FloodFill4(x+1, y, oldColor, newColor)

FloodFill4(x-1, y, oldColor, newColor)

FloodFill4(x, y+1, oldColor, newColor)

FloodFill4(x, y-1, oldColor, newColor)

Scan-line fill : polygon filling and software renderers

1. Find intersection with edges
2. Sort them by x
3. Fill between pairs

Aspect	Bitmap Characters	Outline Characters	Anti-Aliased Characters
Storage	Pixel grid	Mathematical curves	Pixel grid with intensity
Scalability	Poor	Excellent	Good (depends on outline)
Quality	Blocky at large sizes	Smooth and precise	Smooth edges
Speed	Very fast	Slower (needs rasterization)	Moderate
Use Cases	Early systems, fixed-size fonts	Modern typography, scalable UI	High-quality text rendering

Point Scan Conversion

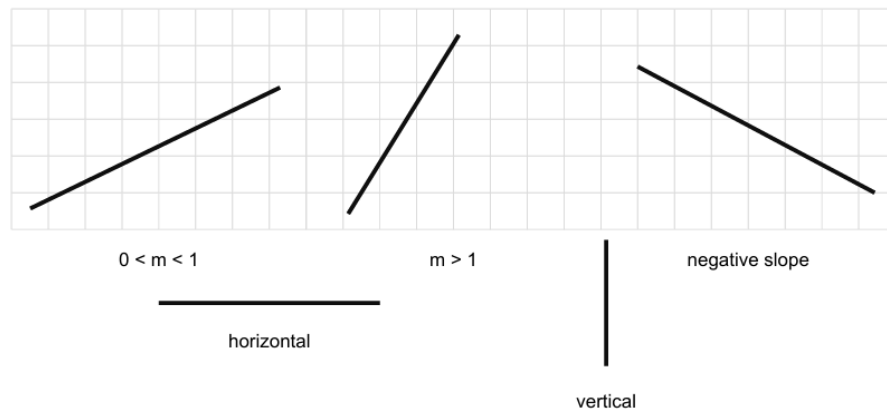
$x_{\text{pixel}} = \text{round}(x)$

$y_{\text{pixel}} = \text{round}(y)$

Line drawing

$$P_1 = (x_1, y_1), P_2 = (x_2, y_2)$$

$$y = mx + b, m = \text{slope} = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$$



Naive line drawing using the line equation

for x from x_1 to x_2 :

$$y = m \cdot x + b$$

plot(x , round(y))

DDA

The **DDA (Digital Differential Analyzer) algorithm** is a simple and widely used method for **scan converting a line** in computer graphics.

DDA_Line(x_1, y_1, x_2, y_2) :

$$dx = x_2 - x_1, dy = y_2 - y_1$$

$$\text{steps} = \max(\text{abs}(dx), \text{abs}(dy))$$

$$x_{\text{inc}} = \frac{dx}{\text{steps}}, y_{\text{inc}} = \frac{dy}{\text{steps}}$$

$$x = x_1, y = y_1$$

for $i = 0$ to steps :

plot(round(x), round(y))

$$x = x + x_{\text{inc}}$$

$$y = y + y_{\text{inc}}$$

Bresenham's line algorithm:

$$E = (x + 1, y)$$

$$NE = (x + 1, y + 1)$$

$$M = (x + 1, y + 0.5)$$

If, $0 \leq m \leq 1, x_1 < x_2$,

Bresenham pseudocode for $(0 \leq m \leq 1)$

Bresenham_Line_0_to_1(x_1, y_1, x_2, y_2) :

$$dx = x_2 - x_1, dy = y_2 - y_1$$

$$p = 2 \cdot dy - dx$$

$$x = x_1, y = y_1$$

plot (x,y)

while $x < x_2$

$$x = x + 1$$

if $p < 0$:

$$p = p + 2 \cdot dy$$

else :

$$y = y + 1$$

$$p = p + 2 \cdot dy - 2 \cdot dx$$

plot(x,y)

Two-Dimensional Transformations

A transformation is a function that maps points from one coordinate space to another.

$$(x, y) \rightarrow (x', y')$$

1. Translation

a. Translation moves every point by the same amount.

$$b. x' = x + t_x$$

$$y' = y + t_y$$

c. It preserves length, angle, shape, orientation, and area.

d. It changes only the position.

e. Translation cannot be represented by 2×2 Matrix

i. Solved by homogenous coordinate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$x' \setminus$$

$$y' \setminus$$

```

1
end{bmatrix} = begin{bmatrix}
1 & 0 & t_x \
0 & 1 & t_y \
0 & 0 & 1
end{bmatrix}
times
begin{bmatrix}
x \
y \
1 \
end{bmatrix}`$

```

1. Translate $(4, 2)$ by $(3, -1)$

$$x' = 4 + 3 = 7$$

$$y' = 2 - 1 = 1$$

1. Rotation

- a. Rotation turns a point around the origin by an angle θ
- b. Usually, Positive rotation is counterclockwise
- c. $x' = x \cos \theta - y \sin \theta$
 $y' = x \sin \theta + y \cos \theta$
- d. Rotate $(5, 2)$ by 90° counterclockwise about the origin
For $90^\circ \rightarrow x' = -y, y' = x$
 $(5, 2) \rightarrow (-2, 5)$

2. Scaling

- a. Scaling changes the size of an object.
- b. $x' = s_x \cdot x$
 $y' = s_y \cdot y$
3. $\begin{matrix}$

```

x' \
y'
end{matrix} = begin{matrix}
s_x & 0 \
0 & s_y
end{matrix}

```

times

$\begin{matrix}$

$x \setminus$

y

$\end{matrix}$

1. **Uniform Scaling:** $s_x = s_y$
2. **Non-uniform scaling:** $s_x \neq s_y$
3. Scaling is usually performed relative to the origin. This means the object may not only change size but also shift position.
4. To scale an object about its own center, we use a composite transformation.
5. Scale (3, 4) by
 $s_x = 2, s_y = 3$
 $x' = 2 \times 3 = 6$
 $y' = 3 \times 4 = 12$

1. Reflection

a. Reflection **produces a mirror image.**

b. Reflection about x axis

i. $x' = x$
 $y' = -y$

2. $\text{\texttt{\text{Matrix}}} = \begin{matrix}$

$\begin{matrix}$

$\begin{matrix}$

$\end{matrix}$

1. Reflection about y axis

1. $x' = -x$
 $y' = y$

1. Shear

a. Shear **slants an object**

2. An x -shear changes x according to y

$$\begin{aligned} x' &= x + s_{h_x}y \\ y' &= y \end{aligned}$$

$\text{\texttt{Matrix}} = \begin{matrix}$

$\begin{matrix}$

$\begin{matrix}$

end{matrix}`\$

1. A y -shear changes y according to x

$$x' = x$$

$$y' = y + sh_y x$$

Homogenous Coordinate

Homogenous coordinates add an extra dimension to represent points, a $2D$ point (x, y) is written as $(x, y, 1)$

- Solved 2×2 problems in translation

1. Translation:

$\begin{matrix}$

$x' \ y' \ 1$

$\end{matrix}$

$= \begin{matrix}$

$1 \ 0 \ t_x \ 0 \ 1 \ t_y \ 0 \ 0 \ 1$

$\end{matrix}$

$\begin{matrix}$

$x \ y \ 1$

$\end{matrix}$ `\$

1. Scaling:

$\begin{matrix}$

$s_x \ 0 \ 0 \$

$0 \ s_y \ 0 \$

$0 \ 0 \ 1$

$\end{matrix}$ `\$

1. Rotation

$\begin{matrix}$

$\cos \theta \ -\sin \theta \ 0 \$

$\sin \theta \ \cos \theta \ 0 \$

$0 \ 0 \ 1$

$\end{matrix}$ `\$

1. X-shear

$\begin{matrix}$

$1 \ sh_x \ 0 \$

$0 \ 1 \ 0 \$

```
0 & 0 & 1
end{bmatrix}`$
```

1. Reflection about x-axis

```
$`begin{bmatrix}
1 & 0 & 0 \
0 & -1 & 0 \
0 & 0 & 1
end{bmatrix}`$
```

Composite transformations

Composite transformations refer to the process of applying **two or more transformations in sequence** to a geometric object. Instead of applying each transformation separately, they are combined into a single transformation matrix using **matrix multiplication**.

Scale → Rotate → Translate separately, we first multiply the matrices together first

$$M = T \times R \times S$$

(Order matters, $T \times R \neq R \times T$)

Then apply the single combined matrix,

$$p' = M \times P$$

Rotation about an arbitrary point

The standard rotation matrix rotates around origin.

$$P = (a, b)$$

For example, a clock hand rotates around its base, not around the global origin.

The trick is

1. translate the pivot point to the origin
2. rotate around the origin
3. translate back

$$M = T(a, b) \times R(\theta) \times T(-a, -b)$$

$$p' = T(a, b) \times R(\theta) \times T(-a, -b) \times P$$

Scaling about a fixed point

$$M = T(a, b) \times S(s_x, s_y) \times T(-a, -b)$$

Aspect	Geometric Transformation	Coordinate Transformation
Definition	Changes the actual position, size, or orientation of the object in the plane.	Changes the coordinate system (axes or reference frame) used to describe the object.
Effect on Object	The object itself moves or reshapes.	The object remains fixed; only the coordinate system changes.
Interpretation	Seen as the object being transformed.	Seen as the viewpoint or reference frame being transformed.
Example (Rotation)	Rotating a triangle 90° counterclockwise around the origin.	Rotating the coordinate axes 90° clockwise, making the triangle appear rotated relative to the new axes.
Mathematical Form	Transformation matrix multiplies the object's coordinates.	Transformation matrix multiplies the coordinate system basis vectors.
Use Case	Used in graphics to animate, move, resize, or reshape objects.	Used in physics/engineering when changing frames of reference (e.g., world vs local coordinates).

Concept	Definition	Example
Local Coordinates	The coordinate system attached to an individual object.	Car defined with center at (0, 0)
World Coordinates	The global coordinate system in which all objects are placed.	Road, ground, sky all share same axes
Instance Transformation	The process of mapping an object from its local coordinate system into the world coordinate system using transformations (translation, rotation, scaling).	Placing multiple trees at different spots in the world

Two-Dimensional Viewing and Clipping

A **window** is a rectangular area defined in the **world coordinate system**.

- It answers the question: *"Which part of the world do we want to see?"*
- Only the objects (or parts of objects) inside this window are considered for display; everything outside can be clipped away.

The viewport is the **rectangular region** on the **display device** where the **window will be shown**.

- *"Where on the screen should we display the selected world region?"*

Window-to-viewport mapping

A point in window is $(x_w, y_w) \rightarrow (x_v, y_v)$

Window $\rightarrow x_{wmin}, x_{wmax}, y_{wmin}, y_{wmax}$

Viewport $\rightarrow x_{vmin}, x_{vmax}, y_{vmin}, y_{vmax}$

$$s_x = \frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}}, s_y = \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}}$$

$$x_v = x_{vmin} + (x_w - x_{wmin}) s_x$$

$$y_v = y_{vmin} + (y_w - y_{wmin}) s_y$$

- **Aspect ratio:** Ratio of the width to height. $100 \times 100 \rightarrow ar : 1 : 1$
- **Avoid distortion:** $s = \min(s_x, s_y)$

Clipping

Clipping **removes parts of objects** that **lie outside the viewing window**.

- **Without Clipping:** A line or polygon outside the visible area may still be processed unnecessarily
- **With clip:** Only visible parts are passed forward
 - To **avoid** drawing **invisible geometry**.
 - To **reduce** unnecessary **computation**.
 - To make sure **objects do not draw outside their allowed** region
 - To **prepare geometry** for later rasterization

- **Types**

- **Point Clipping**

PointClip (x, y) :

if $x_{wmin} \leq x \leq x_{wmax}$ and $y_{wmin} \leq y \leq y_{wmax}$:

accept point

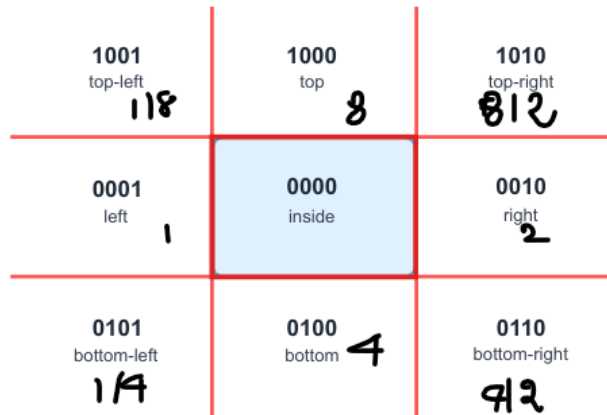
else :

reject point

- **Line Clipping**

- **Cohen–Sutherland algorithm**

- The Cohen–Sutherland algorithm is a classic line clipping algorithm for rectangular windows.
- Its main idea is to assign each endpoint a 4-bit region code describing where the point lies relative to the clipping window.



if $y > y_{\max}$: set TOP bit
 if $y < y_{\min}$: set BOTTOM bit
 if $x > x_{\max}$: set RIGHT bit
 if $x < x_{\min}$: set LEFT bit

■ **Three main cases**

• **Trivial accept**



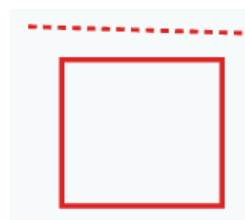
- if both endpoint codes are 0000 then both endpoints are inside
 $\text{code1 OR code2} == 0$

The whole line is accepted.

- OR test checks whether both endpoints are inside

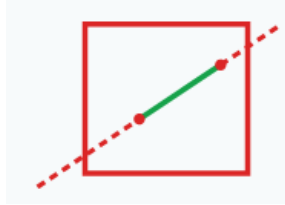
• **Trivial reject**

- If the bitwise AND of the two codes is not zero
 $\text{code1 AND code2} \neq 0$



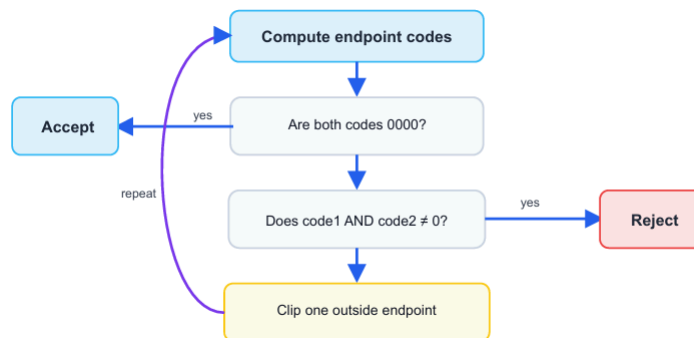
Then both endpoints are outside on the same side of the window. The line is rejected.

- *AND* test checks whether endpoints are on the same outside side.
- **Clip and repeat**
 - If neither of the above cases applies, the line may cross the window.



- We choose one outside endpoint, find its intersection with the relevant window boundary, replace that endpoint with the intersection point, recompute the code, and repeat

■ Algorithm



CohenSutherlandClip (P1, P2):

code1 = ComputeCode(P1)

code2 = ComputeCode(P2)

while true:

if code1 == 0 and code2 == 0 :

accept line

break;

else if code1 and code2 != 0 :

reject line

break

else

choose an endpoint that is outside

find intersection with an appropriate boundary

replace that endpoint with the intersection pointrecompute its code

- **Boundary intersection formulas**

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

Intersection with top boundary, $y = y_{max}$

$$x = x_1 + (y_{max} - y_1)/m$$

$$y = y_{max}$$

Intersection with bottom boundary, $y = y_{min}$

$$x = x_1 + (y_{min} - y_1)/m$$

$$y = y_{min}$$

Intersection with right boundary, $x = x_{max}$

$$y = y_1 + m(x_{max} - x_1)$$

$$x = x_{max}$$

Intersection with left boundary, $x = x_{min}$

$$y = y_1 + m(x_{min} - x_1)$$

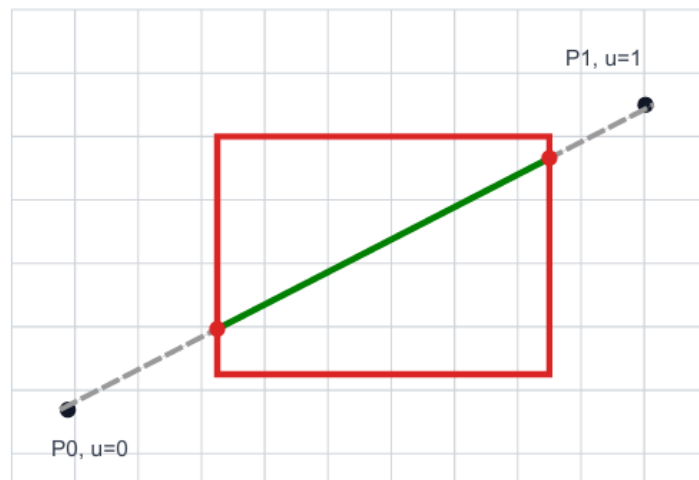
$$x = x_{min}$$

- **Liang-Barsky line clipping**

- Instead of using region codes, it represents the **line segment** parametrically:

$$P(u) = P_0 + u(P_1 - P_0)$$

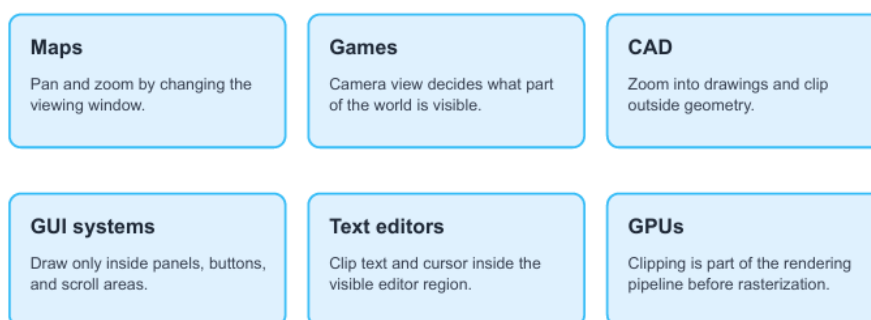
$$\text{where, } 0 \leq u \leq 1$$



- The algorithm finds the interval of u values for which the line is inside the clipping window.

Aspect	Cohen-Sutherland Algorithm	Liang-Barsky Algorithm
Approach	Uses region codes (outcodes) to classify endpoints and applies bitwise logic	Uses parametric line equations and inequalities to directly compute

Aspect	Cohen–Sutherland Algorithm	Liang–Barsky Algorithm
	(AND/OR) to decide trivial accept/reject or partial clipping.	intersection points with clipping window boundaries.
Trivial Cases	Fast trivial accept (both codes = 0000) and trivial reject (bitwise AND $\neq$ 0000).	No trivial accept/reject; relies on parameter checks.
Efficiency	Efficient for trivial cases, but intersection calculations can be slower.	More efficient for intersection calculations since it avoids repeated clipping steps.
Complexity	Conceptually simpler, easier to implement with bitwise logic.	Mathematically more involved (requires parametric equations and inequalities).
Window Shape	Works only with rectangular clipping windows.	Works only with rectangular clipping windows.
Output	Produces clipped line segment after iterative clipping.	Produces clipped line segment directly by solving inequalities.
Use Case	Good for teaching and simple graphics systems; widely used historically.	Preferred in modern systems for efficiency in intersection calculations.



Polygon Clipping, Text Clipping, and the 2D Graphics Pipeline

Polygon Clipping

- Polygon clipping is a process in computer graphics used to cut a polygon so that only the portion inside a defined viewing window is displayed.
- **Differences from line clipping**
 - lose vertices
 - gain new intersection vertices
 - change its shape
 - sometimes break into multiple pieces depending on the polygon and clipping region
- **Sutherland-Hodgeman polygon**

- The **Sutherland–Hodgman algorithm** is a classic method in computer graphics for **polygon clipping**—that is, trimming a polygon so only the part inside a clipping window (usually a rectangle) is kept.
- It clips a polygon against one clipping boundary at a time.
- Divide-and-process style makes the algorithm easier
- **Steps**
 - Start with the original polygon.
 - For each edge of the clipping window (left, right, top, bottom):
 - Take the polygon's vertices one by one.
 - Decide whether each vertex is **inside** or **outside** the clipping boundary.
 - Depending on the case, output either:
 - The vertex itself (if inside),

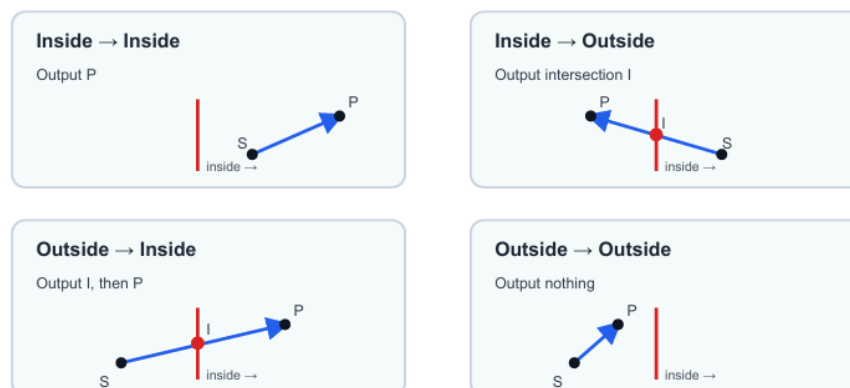
$$x_{min} \leq x \leq x_{max} \text{ and } y_{min} \leq y \leq y_{max}$$
 - The intersection point (if the edge crosses the boundary),
 - Or nothing (if completely outside).
 - The output of one boundary becomes the input for the next boundary.
 - After all four boundaries are processed, the result is the clipped polygon.

```

input polygon
↓ clip against left boundary
intermediate polygon
↓ clip against right boundary
intermediate polygon
↓ clip against bottom boundary
intermediate polygon
↓ clip against top boundary
final clipped polygon

```

- **The four cases in polygon clipping**



- **Sutherland–Hodgman pseudocode**

Sutherland-HodgmanPolygonClip (Polygon, clipWindow)

outputList = polygon

for each clipping boundary edge E:

 inputList = outputList

 outputList = empty

 S = last vertex in inputList

 for each vertex P in inputList :

 if P is inside E:

 if S is inside E:

 output P

 else:

 output intersection(S,P,E)

 output P

 else:

 if S in inside E:

 output intersection(S,P,E)

 else:

 output nothing

 S=P

return outputList

- **Why S = last vertex initially ?**

- A polygon is closed. The edge from the last vertex to the first vertex is also part of the polygon.
- So before processing the first vertex, we treat the previous vertex as the last vertex of the polygon.

- **Limitations and special cases**

- Only work for convex window
- It doesn't work for concave window
 - A simple inside/outside test against each edge may not produce the expected result
 - Weiler–Atherton clipping is used to handle this

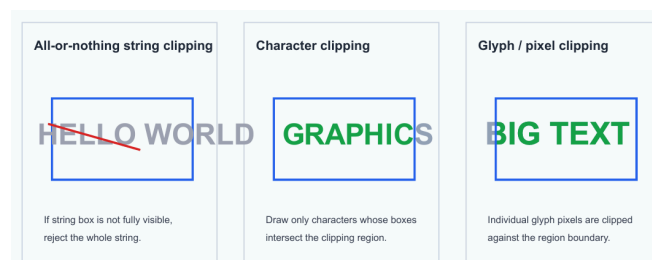
- **Order**

- Clipping should happen before filling
 - If we fill before clipping, the fill operation may affect pixels outside the visible region.

- transform polygon → clip polygon → fill/rasterize polygon

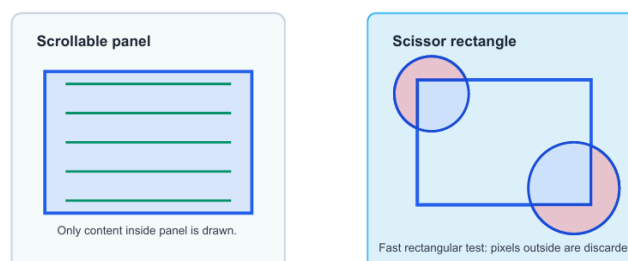
Text Filling

Strategy	How It Works	Pros	Cons	Typical Use Cases
1. All-or-nothing string clipping	The entire string is accepted or rejected based on its bounding box. If any part lies outside, the whole string may disappear.	Very simple to implement, fast.	Crude: even if most of the text is visible, it can vanish completely. Poor user experience.	Early/simple systems, debugging, low-priority text rendering.
2. Character clipping	Each character is tested separately. Characters outside the clipping region are rejected, visible ones are drawn.	Better than all-or-nothing, avoids losing the whole string.	Still rough: characters near the boundary may be cut off abruptly.	Console text rendering, basic GUIs, situations where speed matters more than aesthetics.
3. Glyph / pixel clipping	Each glyph's pixels or vector outline are clipped precisely against the boundary.	Best visual quality, smooth handling of partial characters. Preserves readability.	Computationally more expensive, requires more complex algorithms.	Modern text rendering (word processors, browsers, design software).



Scissor clipping

A very common practical clipping method is rectangular clipping, often called a scissor test or scissor clipping.



Example: A scrollable list may have many items, but only the items inside the panel should be visible.

Three-Dimensional Transformations

A 3D coordinate system has three axes, $P = (x, y, z)$

Right-handed coordinate system

- point the fingers of your right hand from +x toward +y,
- your thumb points toward +z.

Aspect	Point	Vector
Meaning	Location in space	Direction + magnitude
Origin	Fixed in space	Usually relative to origin (0,0,0)
Operations	Can be translated by adding a vector	Can be scaled, normalized, dotted, crossed
Homogeneous coordinates	Represented as (x,y,z,1)	Represented as (x,y,z,0)
Use cases	Vertices, positions	Motion, orientation, lighting calculations

• Translation

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

• Scaling in 3D

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

• Rotation in 3D

Rotation about x-axis

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

Rotation about y-axis

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

Rotation about z-axis

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

X-Axis

$$x' = x$$

$$y' = y \cos\theta - z \sin\theta$$

$$z' = y \sin\theta + z \cos\theta$$

Matrix

$$\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & \cos\theta & -\sin\theta \end{bmatrix}$$

$$\begin{bmatrix} 0 & \sin\theta & \cos\theta \end{bmatrix}$$

Using homogeneous coordinates

$$\begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & \cos\theta & -\sin\theta & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & \sin\theta & \cos\theta & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$$

Y-axis

$$x' = x \cos\theta + z \sin\theta$$

$$y' = y$$

$$z' = -x \sin\theta + z \cos\theta$$

Matrix

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

Homogenous

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -\sin\theta & 0 & \cos\theta & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation around the z-axis

$$x' = x \cos\theta - y \sin\theta$$

$$y' = x \sin\theta + y \cos\theta$$

$$z' = z$$

Matrix:

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \end{bmatrix}$$

$$\begin{bmatrix} \sin\theta & \cos\theta & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

Using homogeneous coordinates

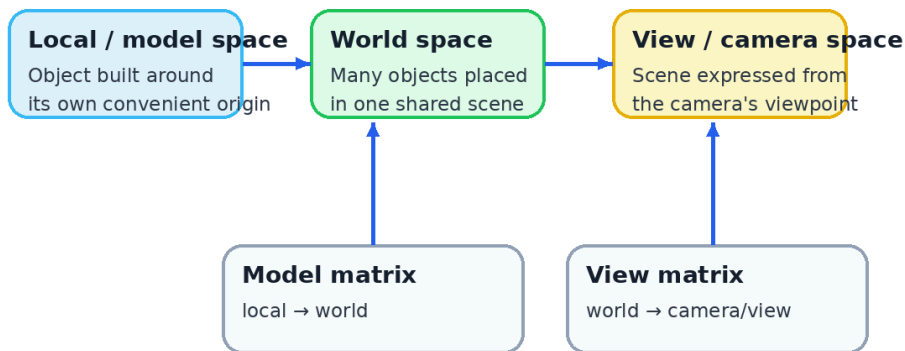
```

[ cosθ  -sinθ   0   0 ]
[ sinθ   cosθ   0   0 ]
[  0      0     1   0 ]
[  0      0     0   1 ]

```

Coordinate spaces in a 3D pipeline

Coordinate Spaces in a 3D Pipeline



A 3D object is usually not modeled directly in screen coordinates. It travels through several coordinate spaces before becoming pixels.

Local or model space: The object is defined in its own coordinate system.

World space: The object is placed into the scene.

View or camera space: The world is described relative to the camera.

Matrix	Purpose	Space Conversion	Key Components	Notes
Model Matrix	Places the object in the scene	Local → World	Translation, rotation, scaling	Each object has its own model matrix; defines its position/orientation relative to the world
View Matrix	Positions and orients the camera	World → Camera (View)	Camera position, target, up vector	Conceptually moves the world so the camera sits at the origin looking down its axis

Matrix	Purpose	Space Conversion	Key Components	Notes
Projection Matrix	Projects 3D scene onto 2D screen	Camera → Clip/Projection	Perspective (FOV, aspect ratio, near/far planes) or Orthographic parameters	Defines how depth and perspective are handled; after this step comes viewport transform

A normal vector is a **vector perpendicular** to a surface. **Normals** are **extremely important** for lighting.

Projection

- Mathematically, projection maps a 3D point to a 2D point:

$$(x, y, z) \rightarrow (x', y')$$

- Types**

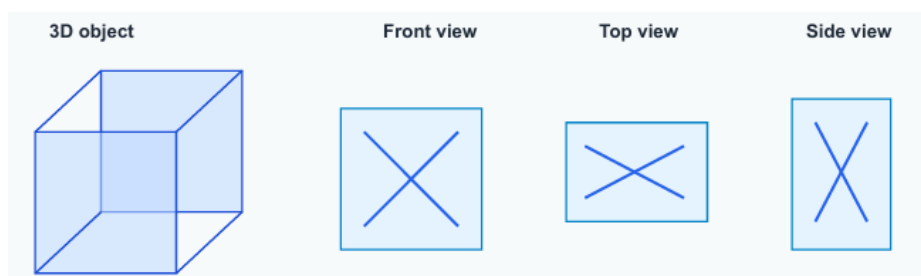
- Parallel Projection**

- In parallel projection, projection lines are parallel to one another.

- Types**

- Orthographic projection**

- Orthographic projection is a type of **parallel projection** where the **projection lines are perpendicular** to the view plane.
 - Used in **engineering and technical** drawing
 - $(x, y, z) \rightarrow (x, y)$



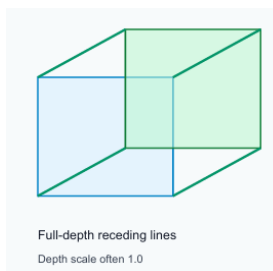
- Why orthographic projection is useful**

- Orthographic projection **does not shrink objects with distance**.
 - This makes it useful for precise measurement.

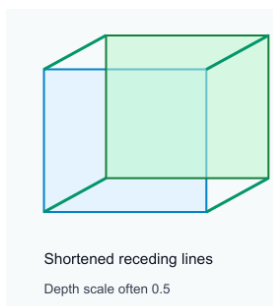
- Application**

- engineering drawings
 - architectural plans

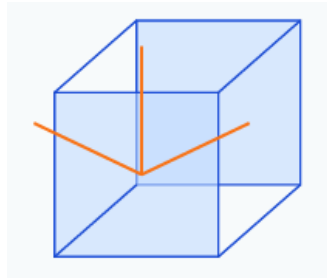
- CAD software
- product design
- machine part diagrams
- technical manuals
- **Oblique projection**
 - The front face of an object is shown **true to shape**, while **depth is represented using angled receding lines**.
 - **Usefulness**
 - Oblique projection is easy **to draw by hand** and can **clearly show the front face** of an object.
 - **Two forms**
 - **Cavalier projection**
 - the receding depth is often drawn at full scale.



- **Cabinet**
 - the receding depth is usually shortened, often by half

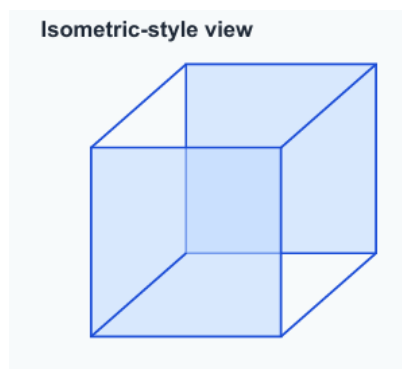


- **axonometric projection**
 - Axonometric projection is a type of **parallel projection where the object is rotated** relative to the projection plane so that **multiple faces are visible**.



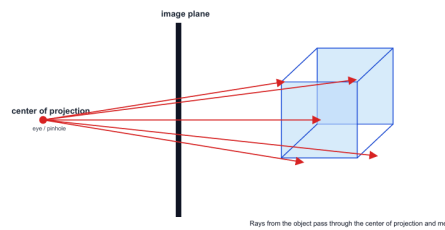
- **isometric projection**

- In **isometric projection**, the three principal axes (x, y, z) are equally foreshortened.
- This gives a balanced 3D-like view without perspective shrinking.

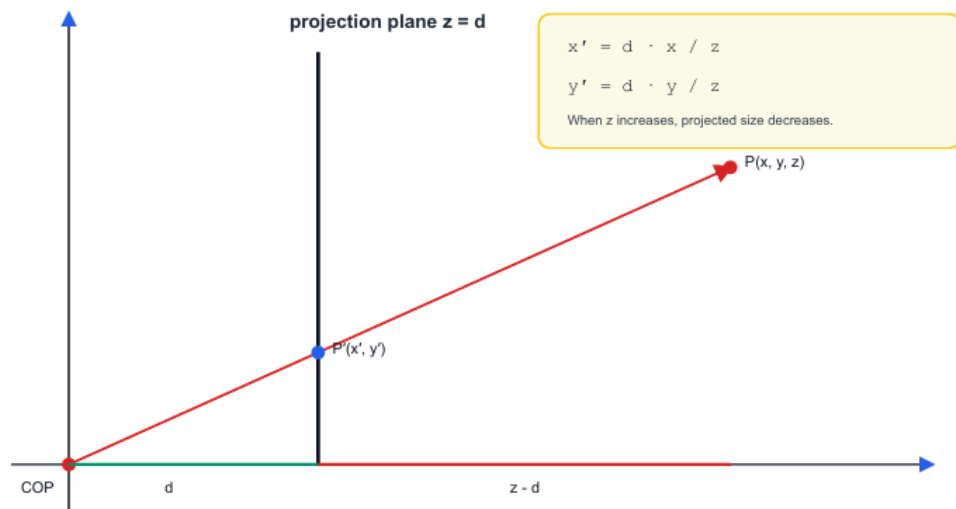


- The angles between the projected axes are all 120° , giving a balanced view of the object.
- **Popular because**
 - Isometric projection is visually clear and easy to understand.
- **Characteristics**
 - Equal scaling
 - No perspective shrinking
 - Multiple faces visible
- **Application**
 - technical illustrations
 - game art
 - diagrams
 - assembly instructions
 - architectural concept drawings
- **Perspective Projection**
 - **Perspective** projection lines meet at a **single point** called **the center of projection**.

- This is similar to a **pinhole camera**.
- Objects farther away appear smaller.

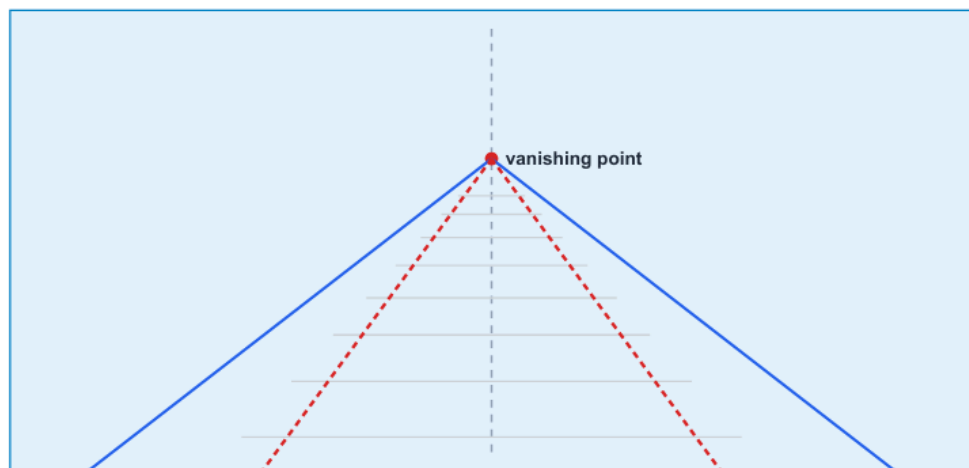


- $x' = dx/z$
 $y' = dy/z$
- $x_{\text{screen}} = x_{\text{clip}}/w_{\text{clip}}$
 $y_{\text{screen}} = y_{\text{clip}}/w_{\text{clip}}$
 $z_{\text{screen}} = z_{\text{clip}}/w_{\text{clip}}$
- $w_{\text{clip}} = z$
- Projection is one of the strongest reasons homogeneous coordinates are used in computer graphics.
 - They allow perspective projection to be represented within a matrix pipeline.
- **Terms**
 - **Center of projection**
 - The point through which **projection rays pass**. In a camera analogy, this is the **pinhole or eye**.
 - **Projection Plane**
 - The plane where the **image is formed**
 - **Projectors**
 - The lines **connecting 3D points** to the **center of projection**.
- **Perspective formula**



■ Vanishing points

- A vanishing point is a point in the image **where parallel lines in 3D appear to meet.**



■ Types

- one-point perspective
- two-point perspective
- three-point perspective

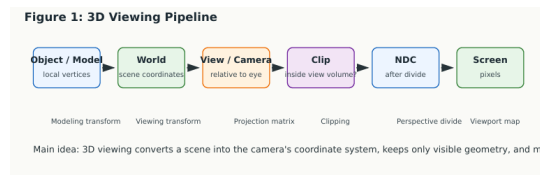
Perspective Type	Vanishing Points	Description	Common Examples	Extra Notes
One-point perspective	1 (usually on the horizon line)	One major direction of the object recedes into depth. Front face is parallel to the picture plane.	Looking straight down a road, into a corridor, or at a box front-on with depth lines receding.	Simplest form; often used in interior drawings and beginner exercises. Creates a strong sense of depth along a single axis.

Perspective Type	Vanishing Points	Description	Common Examples	Extra Notes
Two-point perspective	2 (both on the horizon line)	Two major horizontal directions recede to two different vanishing points. Vertical lines remain parallel.	Viewing the corner of a building, a box rotated so two sides are visible.	More dynamic than one-point; commonly used in architectural sketches. Shows width and depth realistically.
Three-point perspective	3 (two on horizon, one above or below)	Vertical lines also converge to a third vanishing point.	Looking up at a tall skyscraper, or looking down from a high height.	Adds dramatic realism. Often used in comics, cinematic shots, and exaggerated viewpoints. Can feel more immersive but harder to construct.

Feature	Parallel Projection	Perspective Projection
Projection lines	Remain parallel	projection lines meet at a center of projection
Appearance of distant objects	Same size as near objects	Appear smaller as they move farther away
Realism	Less realistic	More realistic, mimics human vision
Measurement accuracy	Preserves true dimensions, easier for technical work	Distorted by depth, measurements not preserved
Common use	CAD, engineering drawings, architectural plans	Games, films, visualization, VR
Computation	Simpler, less math-intensive	More complex, requires perspective divide
Depth perception	No depth cues (flat look)	Strong depth cues, sense of distance
Visual effect	Useful for precision and clarity	Useful for immersion and realism

Three-Dimensional Viewing and Clipping

Coordinate Spaces in the 3D Pipeline



1. Object or model coordinates

- There are local coordinates used when an object is created

2. World Coordinates

- Object coordinates are transformed into a shared scene coordinate system.
- All objects are placed relative to one another.

3. View or camera coordinates

- Object coordinates are transformed so that the camera becomes the reference frame.
- In this coordinate system, the camera is usually placed at the origin and looks along a standard direction.

4. Clip Coordinates

- After projection, points are represented in homogenous clip coordinates.
- Clipping is usually performed in the space.

5. Normalized Device Coordinates

- After the perspective divide, visible points are mapped to a normalized cube, often with x, y, z values in a standard range.

6. Screen or Window Coordinates

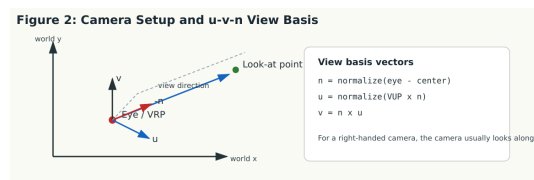
- Normalized coordinates are mapped to actual screen pixels.

The camera analogy

A virtual camera is similar to a real camera, but it defined mathematically.

3 things

- Eye Position
- Look at point (center)
- View up vector (VUP)



u : camera's local right direction

v : camera's local up direction

n : camera's backward direction

- **View Coordinate System**

$$n = \frac{\text{eye} - \text{center}}{\|\text{eye} - \text{center}\|}$$

$$u = \frac{\text{VUP} \times n}{\|\text{VUP} \times n\|}, \text{ Cross product}$$

$$v = n \times u, \text{ cross product}$$

- **World-to-View Transformation**

World-space points are converted to view-space points

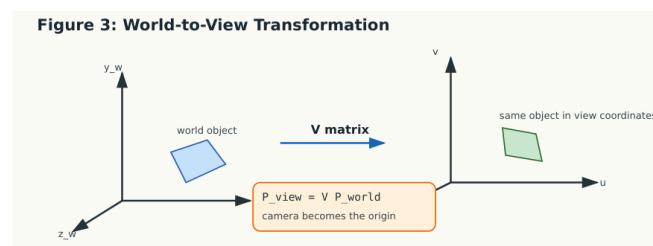
using column vector convention,

$$P_{\text{view}} = V P_{\text{world}}$$

where V is the view matrix.

A common view matrix has the form:

$$V = \begin{bmatrix} u_x & u_y & u_z & -u \cdot \text{eye} \\ v_x & v_y & v_z & -v \cdot \text{eye} \\ n_x & n_y & n_z & -n \cdot \text{eye} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



View Volume

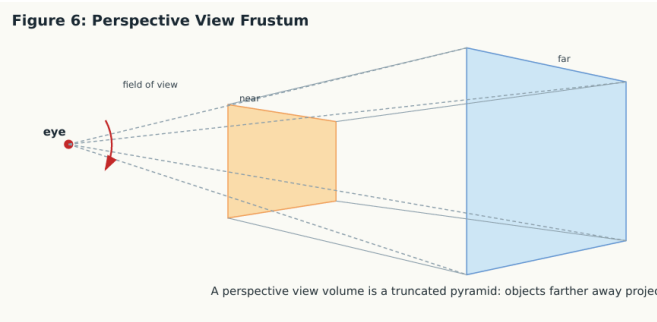
The view volume is the region of 3D space visible to the camera.

1. Orthographic view volume

a. In orthographic projection, projectors are parallel. The view volume is a rectangular box.

2. Perspective View Frustum

a. In perspective projection, all **projectors** meet at the **eye point**. The view volume is a truncated pyramid called a **frustum**.



3. Canonical View Volume

- Different cameras can have different positions, directions, field of view values, aspect ratios and near/far distances.
- Clipping directly in every possible frustum would be inconvenient. So graphics systems often transform the view volume into a standard shape called the canonical view volume.

$$\begin{aligned} -1 &\leq x \leq 1 \\ -1 &\leq y \leq 1 \\ -1 &\leq z \leq 1 \end{aligned}$$

Clipping

1. Point Clipping

- A point (x, y, z) is inside if
 - $-1 \leq x \leq 1$
 - $-1 \leq y \leq 1$
 - $-1 \leq z \leq 1$

2. Clipping Plane and Inside Test

- A clipping plane can be written as

$$Ax + By + Cz + D = 0$$
- For a point $P(x, y, z)$, compute:
 - $f(p)$
- The sign of $f(p)$ tells which side of the plane the point lies on

3. Line Clipping in 3D

- Two points can be written

$$P(t) = P_0 + t(P_1 - P_0)$$

$$0 \leq t \leq 1$$
- If a line crosses a clipping plane, the intersection point occurs where

$$Ax(t) + By(t) + Cz(t) + D = 0$$

c. A compact formula is,

$$t = \frac{-f(P_0)}{f(P_1) - f(P_0)}$$

where,

$$f(P) = Ax + By + Cz + D$$

For a line segment

1. If both endpoint outcodes are 000000, the line is completely inside.
2. If the bitwise AND of the two outcodes is nonzero, the line is completely outside on the same side of at least one plane
3. Otherwise, the line may cross the view volume and must be clipped.

• Polygon Clipping in 3D

- A common approach is similar to the **Sutherland-Hodgman** algorithm:
 1. Clip the polygon against one plane.
 2. Use the result as input for the next plane.
 3. Repeat for all six clipping planes.

Start vertex	End vertex	Output
inside	inside	end vertex
inside	outside	intersection point
outside	inside	intersection point and end vertex
outside	outside	nothing

• Why Clip Before the Perspective Divide?

- Points are in homogenous clip coordinates:
$$P_{dip} = (x_c, y_c, z_c, w_c)$$
- To get normalized device coordinates, we divide by w_c

$$x_{ndc} = \frac{x_c}{w_c}, y_{ndc} = \frac{y_c}{w_c}, z_{ndc} = \frac{z_c}{w_c}$$

However, if geometry crosses the near plane or goes behind the camera, division by w_c can create invalid or unstable results

Therefore, clipping is normally performed before the perspective divide.